

GREEKS ENGINE

Option Greeks in one hardware pass

A fixed-point FPGA datapath for the Heston model that computes the price and all nine sensitivities with reverse-mode algorithmic differentiation, verified bit-exact in simulation and on a Zynq-7020 board

Project report and status

Yash Agrawal · BITS Pilani

29 September 2026 · branch pipeline-shared-mult

Abstract

Risk management of option portfolios needs not only prices but their sensitivities to every input, the Greeks. Under the Heston stochastic-volatility model the price has no closed form and is computed numerically, here by the Fourier-cosine (COS) method with 128 terms. The usual way to obtain Greeks from such a pricer, bump-and-reprice, costs 19 pricings for nine Greeks and its accuracy depends on an unsafe choice of bump size. Reverse-mode algorithmic differentiation (AAD) obtains all Greeks from one backward sweep, and is standard in software.

This project builds AAD as a hardware datapath. The Heston-COS price and its adjoint sweep are written once as an operation graph, from which a generator derives a bit-accurate emulator, a double-precision evaluation, a first-order error bound for every output, and a fixed-point, resource-shared, modulo-scheduled Verilog datapath. On a Zynq-7020-sized configuration the price and nine Greeks take 4,733 clock cycles, 1.04 times the cost of the price alone and 18.4 times fewer cycles than bump-and-reprice on identical hardware. Measured error never exceeds 0.103 of the guaranteed bound on a 21-case grid. The design routes on a Zynq-7020 at about 79 MHz (57.8 μ s per evaluation) and, on a ZedBoard, 50 random cases run back to back gave results bit-identical to the emulator. Pricing calls through put-call parity, found during the board work, cut the worst call price error 75-fold.

Contents

Abstract	2
1 Motivation	4
2 Related work and novelty	4
3 Aim and contributions	5
4 Architecture	5
4.1 One description, five uses	5
4.2 Shared arithmetic and the modulo schedule	6
4.3 Host split and board interface	6
4.4 Calls through put-call parity	6
5 Verification method	6
6 Results	7
6.1 Speed	7
6.2 Accuracy	8
6.3 Calls through put-call parity	9
6.4 Implementation on the Zynq-7020	10
6.5 On silicon	11
6.6 Generation 1: the hand-written engine	11
7 Work done	12
8 Limitations	13
9 Next steps	13
10 Conclusion	14

Reproducing the results 14

1 Motivation

A bank holding options must know, continuously, how each price responds to spot, strike, maturity, interest rate and every parameter of its pricing model. These first derivatives, the Greeks, drive hedging and risk limits. For the Heston model, which lets volatility follow its own random process, the price has no closed form. It is computed by numerical integration; this project uses the COS method, which expands the payoff in a cosine series and needs only the model's characteristic function.

Three problems motivate the work:

- **Bump-and-reprice is expensive.** Nine Greeks by central differences cost 19 pricings. On the hardware built here that is 86,860 cycles against 4,733 for AAD.
- **Bump-and-reprice is inaccurate.** Its error is U-shaped in the bump size h : rounding dominates when h is small, truncation when h is large. There is no safe choice, and in fixed point the rounding floor is high.
- **AAD is fast and exact, but only in software.** Reverse-mode AAD returns every Greek from one backward sweep at a small multiple of one pricing, typically 3–5 in software because of tape traffic. No published FPGA implementation of it for option Greeks was found (section 2).

2 Related work and novelty

A literature check made on 29 September 2026 (docs/novelty_assessment.md) looked for anything that would make this work not new. It found three separate bodies of work and nothing joining them.

AAD for Greeks in software. Adjoint pathwise Monte Carlo (Giles and Glasserman, Risk 2006) and its implementation by algorithmic differentiation (Capriotti, J. Comput. Finance 14(3), 2011) made AAD the standard way to compute Greeks; production practice is described by Savine (Wiley, 2018). The 2024 review of the field by Capriotti and Giles cites no FPGA or hardware AAD, and nothing on COS or Fourier AAD. AAD has been accelerated on GPUs (Gremse et al., Comput. Phys. Commun. 2016), and Capriotti's US patent 9,058,449 (Credit Suisse, 2015) claims an adjoint Greeks simulator built from general-purpose processors. For Heston specifically, Cui, del Baño Rollin and Germano (EJOR 2017) derive the gradient of the Fourier price analytically.

Greeks and Heston pricing on FPGAs. A 2024 survey of 99 FPGA option-pricing studies (O Mahony et al., Electronics 13(16)) never mentions automatic differentiation, adjoints or Fourier methods. The closest FPGA Greeks work (Klaisonngnoen et al., HEART 2022 and H2RC 2022) runs the STAC-A2 Heston Monte Carlo workload and does not use adjoints. AMD's Vitis Quantitative Finance Library computes Heston Greeks on FPGA by central finite differences, re-running Monte Carlo per bumped parameter, and prices the Heston closed form by trapezoidal integration without Greeks. The COS method has run on GPUs (Zhang and Oosterlee, 2009) but no FPGA implementation was found. An automatic generator of FPGA pricing accelerators exists (Pham, Aung and Kumar, ReConFig 2016), without Greeks.

Reverse mode and precision analysis. Reverse-mode differentiation was introduced to expand accumulated rounding error (Linnainmaa, BIT 1976), and AD has been used to choose FPGA word lengths (Gaffar, Mencer, Luk et al., FPT 2002). Every neural-network training accelerator executes backpropagation, which is reverse mode on a fixed graph.

What is new here. To our knowledge this is the first FPGA implementation of AAD for option Greeks and the first FPGA implementation of the COS method. The individual techniques (static adjoints, modulo scheduling, first-order rounding analysis) are established; the contribution is

their combination for a fixed-point, complex-valued pricing datapath, with a per-Greek error bound, demonstrated bit-exact on silicon. Unpublished industry work cannot be ruled out, so the claim is stated as "to our knowledge".

Work	Method	Platform	Greeks by
Giles & Glasserman 2006; Capriotti 2011	Monte Carlo	CPU	Adjoint / AD
Gremse et al. 2016	General	GPU	AAD
Cui et al. 2017	Heston, Fourier	CPU	Hand-derived gradient
Klaisoongnoen et al. 2022	Heston Monte Carlo (STAC-A2)	FPGA	Not adjoint
AMD Vitis library	Heston Monte Carlo; Fourier (price only)	FPGA	Finite differences
Zhang & Oosterlee 2009	COS	GPU	None (price only)
This work	Heston, COS	FPGA, on silicon	AAD, static datapath, error bound

Table 1. Position relative to the closest prior work.

3 Aim and contributions

Aim. Build a generator that turns the Heston-COS price and its adjoint sweep into a single fixed-point, resource-shared, modulo-scheduled Verilog datapath that fits a Zynq-7020, is provably accurate for every output, and runs on a real board.

	Claim	Evidence
N1	AAD with no tape. The COS pricer performs the same operations for every input, so the reverse sweep is unrolled at design time into a fixed datapath: no tape memory, write port or replay sequencer.	RTL bit-exact on 5 designs; 50 cases on silicon. Static adjoints of fixed graphs are known in AD and ML hardware; the new step is applying them to this pricer.
N2	Nine Greeks for 1.04 pricings. Iterative CORDIC units set the pace, leaving multiplier slots idle that the adjoint's extra multiplies fill.	4,733 vs 4,568 cycles on the Zynq-7020 design; 1.55× on the 64-bit design, where multipliers bind.
N4	A guaranteed error bound for every Greek , computed by applying reverse-mode differentiation to the hardware's own rounding model.	Error ≤ 0.103 of the bound on 21 cases, ≤ 0.199 on 50. The method is Linnainmaa's (1976); its application to an adjoint datapath is new.
N3, N5	Forward and reverse passes share reciprocals; every transcendental is reduced to a countable number of shared multiplies (250 per COS term).	Per-term operation bill; schedule verification.
N6, N7	One description yields emulator, error bound and RTL, which cannot drift; the baseline is bump-and-reprice on the same hardware.	Byte-identical regeneration check; 18.4× on identical hardware.

Table 2. Contribution claims. N1, N2 and N4 carry the paper.

4 Architecture

4.1 One description, five uses

The price and its reverse-mode adjoint are written once as an operation graph in `hardware/gen/heston.py`. The graph is split into a *setup* part evaluated once per pricing (truncation range, grid spacing, characteristic-function constants), a *term* part evaluated for each of the 128 COS terms and accumulated, and a *finish* part (discounting and the final chain rule). From it the generator produces a bit-accurate emulator, a double-precision evaluation,

the first-order error bound, the scheduled RTL, and testbenches that check every register at the cycle it becomes valid.

4.2 Shared arithmetic and the modulo schedule

There are no dividers. Reciprocals and inverse square roots use Newton-Raphson iterations on a normalised mantissa seeded from an 8-bit table; exp uses Tang's method; log a table plus a short series; sine and cosine a CORDIC. One COS term then costs exactly 250 multiplies, 3 CORDIC rotations and 2 CORDIC vectorings, and a modulo scheduler overlaps consecutive terms on shared units. A new term starts every II cycles (the initiation interval). On the Zynq-7020 configuration (56-bit words, 28 fractional bits, 8 multipliers, 3 rotation and 2 vectoring units), all three resource classes give $II = 32$: $\lceil 250/8 \rceil = 32$ for multipliers and $3 \cdot 32/3 = 32$ for iterative CORDICs. Cycles per evaluation are $S + 127 \cdot II + L + F + 2$, with setup length S, term latency L and finish length F; for this design $113 + 4,064 + 506 + 48 + 2 = 4,733$.

4.3 Host split and board interface

The configuration built for the board (heston_aad_z7h) moves setup and finish to the host processor, leaving the 128-term loop on the FPGA: 4,572 cycles. The host supplies 16 precomputed constants. An AXI4-Lite register file (51 input words, a control and a status register, 9 result pairs) makes the core reachable from a Zynq; the 1,456-bit AXI4-Stream alternative is wider than any Zynq port. A sticky range_err output flags any input outside the verified domain instead of returning a wrong value.

4.4 Calls through put-call parity

Priced directly, a call's COS coefficients contain e^b for the upper truncation bound b, which grows large over a wide range and then cancels; in fixed point this lost up to $2.3 \cdot 10^{-3}$ on the price. The term loop now always computes the put, and the finish step adds $S_0 - K e^{-rT}$ and its derivatives in S_0 , K, T and r when a call is requested (section 6.3).

5 Verification method

- **Bit-exact RTL.** Icarus Verilog simulations compare 1,213 to 1,962 registers per design against the emulator at the cycle each becomes valid, plus all outputs; measured cycle counts equal the scheduler's prediction.
- **22 automated checks** in `verify_all.sh`: arithmetic unit tests, both hardware generations, all generated designs and wrappers, the host split, out-of-domain detection, a byte-for-byte check that committed RTL equals fresh generator output, and the error bound. All pass.
- **Accuracy** against two independent double-precision references: the COS method itself (the algorithm the hardware implements) and adaptive quadrature of the Heston Fourier integral (the model itself).
- **On silicon** over JTAG: inputs written, engine started, done polled, all results read and compared bit for bit. The test script was itself tested against a software model of the board with injected faults.

6 Results

6.1 Speed

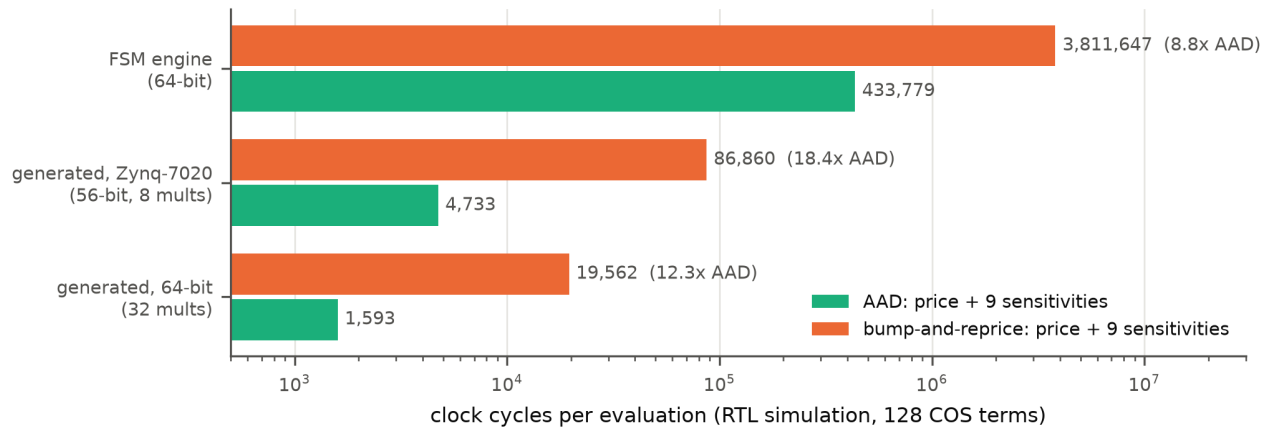


Figure 1. Clock cycles for price + 9 Greeks, AAD against bump-and-reprice on the same hardware (RTL simulation, 128 COS terms, log scale).

Configuration	Multipliers	II	AAD	Price only	Bump	Bump ÷ AAD
Zynq-7020 (56/28)	2	125	16,754	8,700	165,368	9.9×
Zynq-7020 (56/28)	4	63	8,694	4,613	87,715	10.1×
Zynq-7020 (56/28)	8	32	4,733	4,568	86,860	18.4×
Zynq-7020 (56/28)	16	32	4,621	4,563	86,765	18.8×
64-bit (64/32)	1	250	32,530	16,773	318,755	9.8×
64-bit (64/32)	2	125	16,758	8,701	165,387	9.9×
64-bit (64/32)	4	63	8,700	4,636	88,152	10.1×
64-bit (64/32)	8	32	4,709	2,573	48,955	10.4×
64-bit (64/32)	16	16	2,614	1,540	29,328	11.2×
64-bit (64/32)	32	8	1,593	1,026	19,562	12.3×
64-bit (64/32)	64	4	1,073	888	16,940	15.8×
64-bit (64/32)	128	3	942	888	16,940	18.0×

Table 3. Cycles per evaluation against the number of shared multipliers (validation/results/gen_cycles.csv). Highlighted: the configurations used. Generation 1, the hand-written FSM engine, needed 433,779 cycles (3,811,647 for bump-and-reprice).

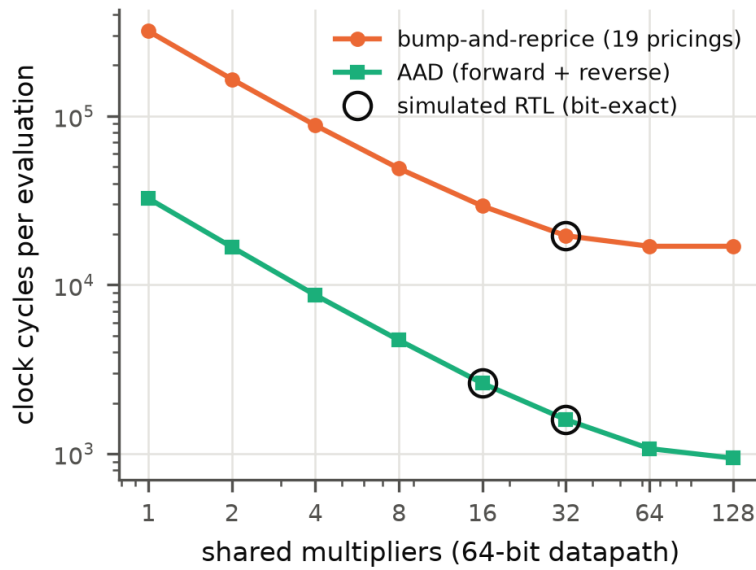


Figure 2. Cycles against shared multipliers, 64-bit datapath. Circled points were simulated as RTL and matched the scheduler exactly.

On the Zynq-7020 configuration a 16th multiplier buys only 2% because II is already pinned at 32 by the CORDIC units. The advantage over bump-and-reprice exceeds the $\sim 9.6\times$ ratio of work (2,394 against 250 multiplies per term) because the price-only pricer leaves multiplier slots idle that AAD fills.

6.2 Accuracy

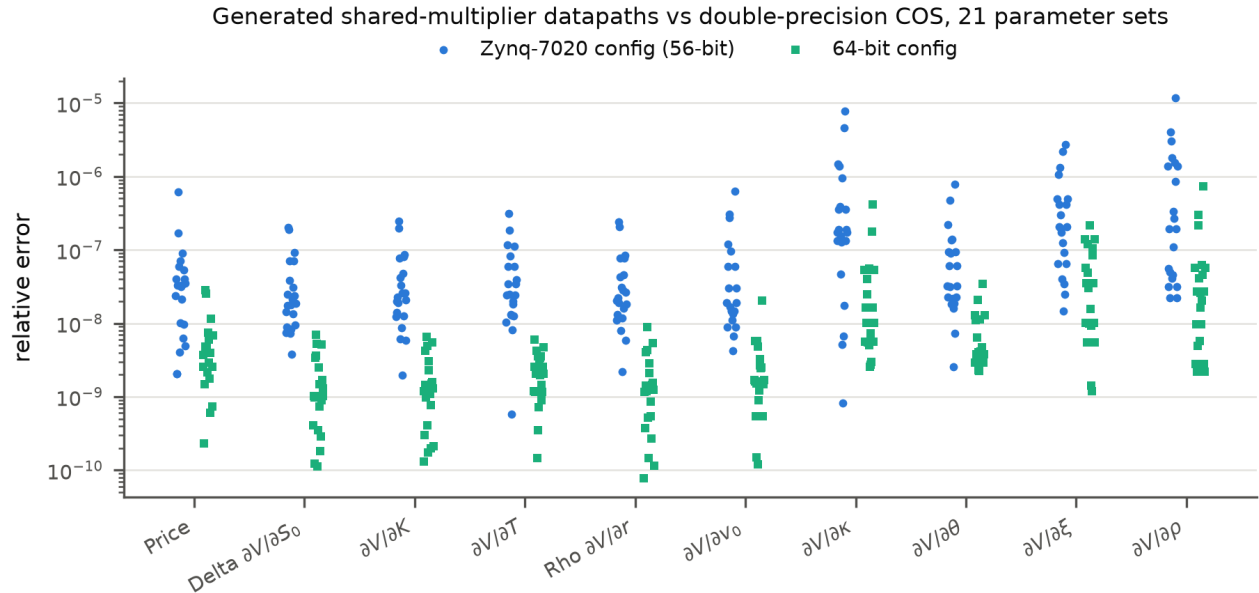


Figure 3. Relative error of the generated designs against the double-precision COS reference over 21 parameter sets (moneyness, maturity, vol-of-vol, correlation; calls and puts).

Output	Zynq-7020 worst rel. error	64-bit worst rel. error	Worst error ÷ bound
Price	6.1e-07	2.8e-08	0.074
Delta $\partial V/\partial S_0$	2.0e-07	6.9e-09	0.049
$\partial V/\partial K$	2.5e-07	6.6e-09	0.051
$\partial V/\partial T$	3.1e-07	6.0e-09	0.093
Rho $\partial V/\partial r$	2.4e-07	8.9e-09	0.049
Vega $\partial V/\partial v_0$	6.2e-07	2.0e-08	0.103
$\partial V/\partial \kappa$	7.8e-06	4.1e-07	0.098
$\partial V/\partial \theta$	7.9e-07	3.4e-08	0.102
$\partial V/\partial \xi$	2.7e-06	2.2e-07	0.099
$\partial V/\partial \rho$	1.2e-05	7.4e-07	0.087

Table 4. Worst relative error over the 21-case grid, and the largest measured error as a fraction of the first-order bound (validation/results/gen_accuracy.csv). Every ratio is below 1: the bound held.

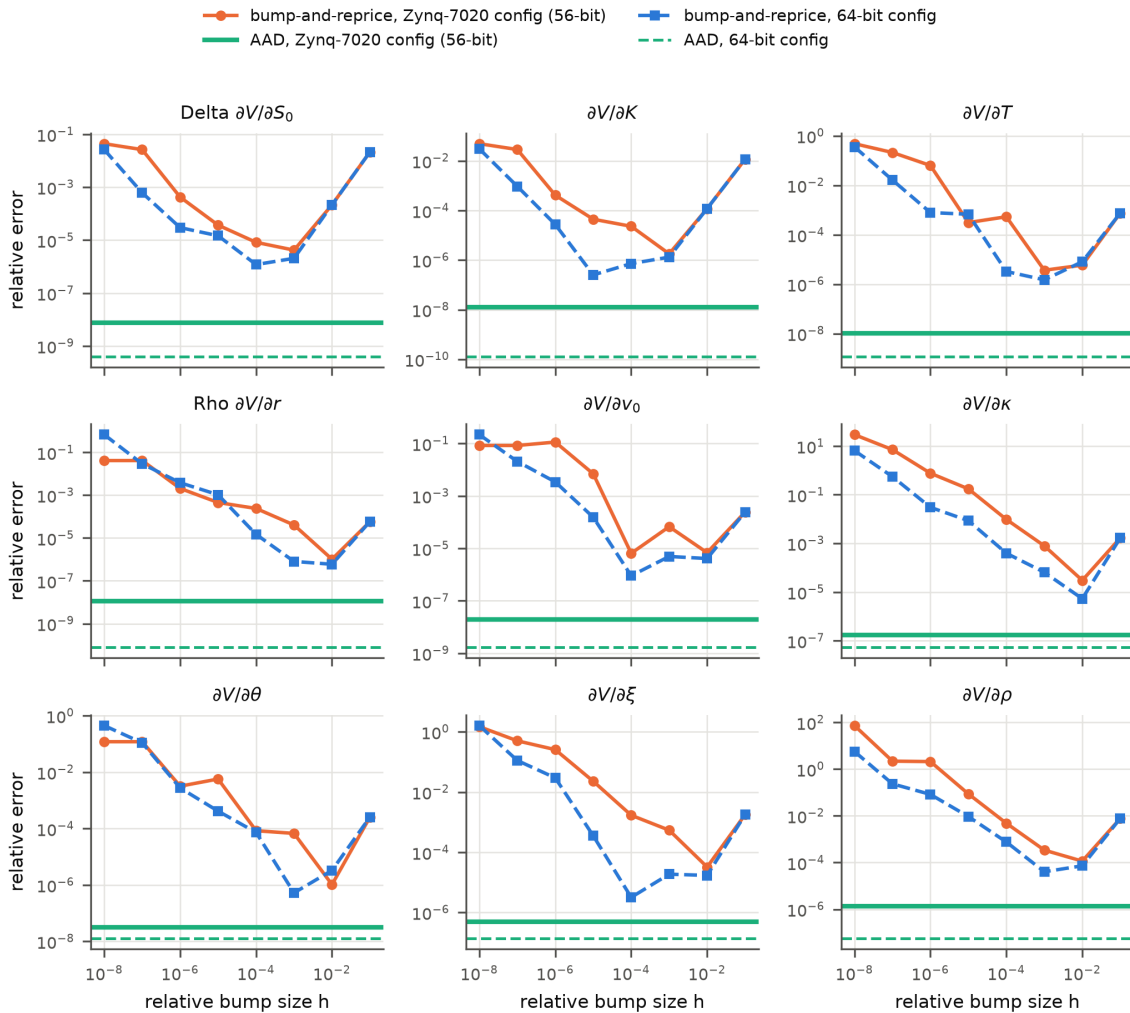


Figure 4. Every Greek: bump-and-reprice error against bump size (curves) and AAD (flat lines), on the same pricer and word length. AAD beats the best bump size chosen in hindsight in every panel.

6.3 Calls through put-call parity

The 50-case board sweep (section 6.5) exposed calls carrying about 180 times the rounding error of puts. Scored against the independent Fourier-integral price on the 22 calls of that sweep:

Worst absolute error, 22 calls	Direct call	Put + parity	Gain
Price	2.3e-3	3.1e-5	75×
Price, median	4.8e-5	5.4e-7	~90×
$\partial V/\partial \kappa$	2.4e-3	3.9e-4	6×
$\partial V/\partial \theta$	1.7e-2	2.5e-3	7×
$\partial V/\partial \xi$	1.3e-2	1.8e-3	7×
$\partial V/\partial \rho$	8.3e-3	4.7e-4	18×
Delta, $\partial V/\partial K$, $\partial V/\partial T$, rho, vega			2–2.5×

Table 5. Effect of put-call parity. Cost: 4,731 $\rightarrow$ 4,733 cycles (Zynq-7020), 1,599 $\rightarrow$ 1,593 (64-bit), one multiply per term fewer. With parity the fixed-point error matches the double-precision COS method's own error, so rounding is no longer the limit. All 22 verification checks pass before and after.

6.4 Implementation on the Zynq-7020

Build	LUT	FF	DSP	Clock	WNS	Power
Engine alone, out of context, xc7z020clg400-1	36,499 (68.6%)	28,559 (26.8%)	72 (32.7%)	100 MHz	-2.639 ns	0.258 W
ZedBoard system, xc7z020clg484-1	38,358 (72.1%)	32,206 (30.3%)	72 (32.7%)	70 MHz	+0.404 ns	0.341 W

Table 6. Vivado 2025.2 results (validation/results/vivado/). Fmax is about 79 MHz alone and 72 MHz inside the board system, limited by 64-bit carry chains and multiplier outputs. Power is Vivado's vectorless estimate; 0.121 W of the board figure is the clock generator. Both builds predate put-call parity.

At the measured 79.1 MHz one evaluation of the host-split design takes 57.8 μ s (65.3 μ s at the board's 70 MHz), and vectorless energy is at most 14.9 μ J. Vivado's 67 DRC warnings contain no errors: 56 suggest using the DSP blocks' internal output registers, the most direct route to 100 MHz.

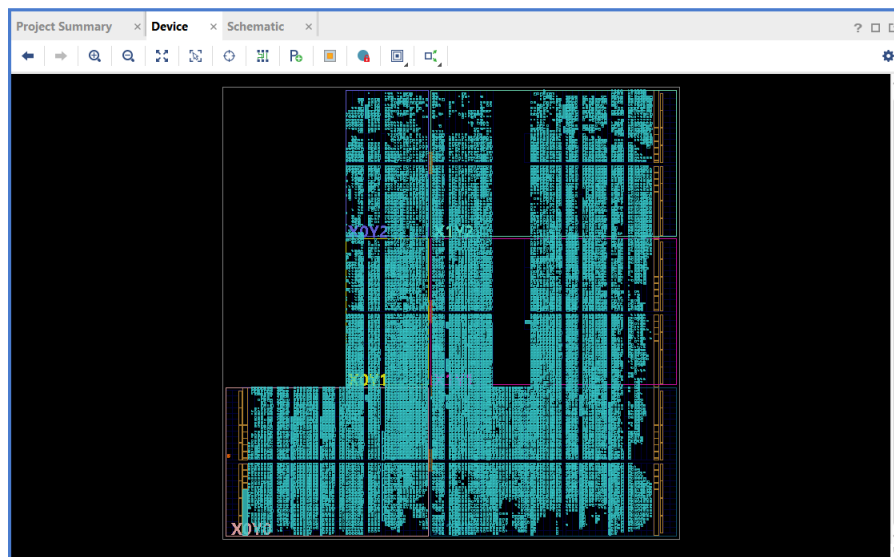


Figure 5. The placed design fills most of the Zynq-7020's programmable logic (Vivado device view).

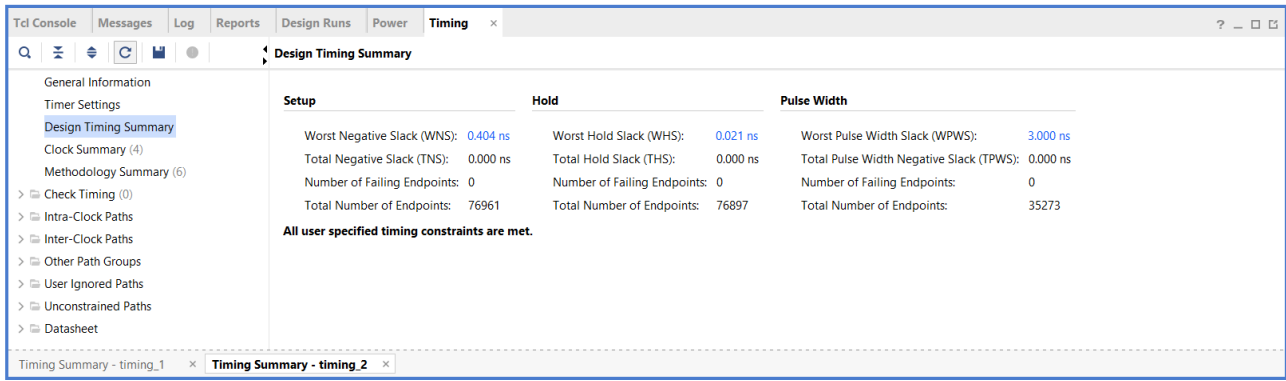


Figure 6. Timing summary of the ZedBoard build: all constraints met at 70 MHz, 0 failing endpoints.

6.5 On silicon

The engine was driven over the ZedBoard's JTAG cable through a JTAG-to-AXI master, with no processor software. After one test vector (9 of 9 results bit-exact), a sweep of 50 cases ran back to back without a reset: the two reference cases plus 48 random draws over the verified domain, 22 calls and 28 puts, from a random seed the hardware's shifter ranges were not fitted on.

Run	Cases	Bit-exact results	range_err	Worst error ÷ bound	Wall time
First vector	1	9 / 9	0	0.031	–
Sweep	50	450 / 450	0	0.268	56.9 s

Table 7. ZedBoard results, 29 September 2026 (validation/results/board_*). Nearly all of the 56.9 s is JTAG register traffic; each case computes in about 65 μ s. These runs used the bitstream built before put-call parity; the re-run with the current design is predicted to reach a worst error ÷ bound of 0.199.

6.6 Generation 1: the hand-written engine

The first engine, a hand-written finite-state machine in 64-bit fixed point, established the error model and the bump-and-reprice comparison. It was accurate (price $3.0 \cdot 10^{-7}$, Greeks $\leq 5.4 \cdot 10^{-5}$) but took 433,779 cycles, 80% of them in 28 bit-serial divisions per term, and needed far more multipliers than a Zynq-7020 has.

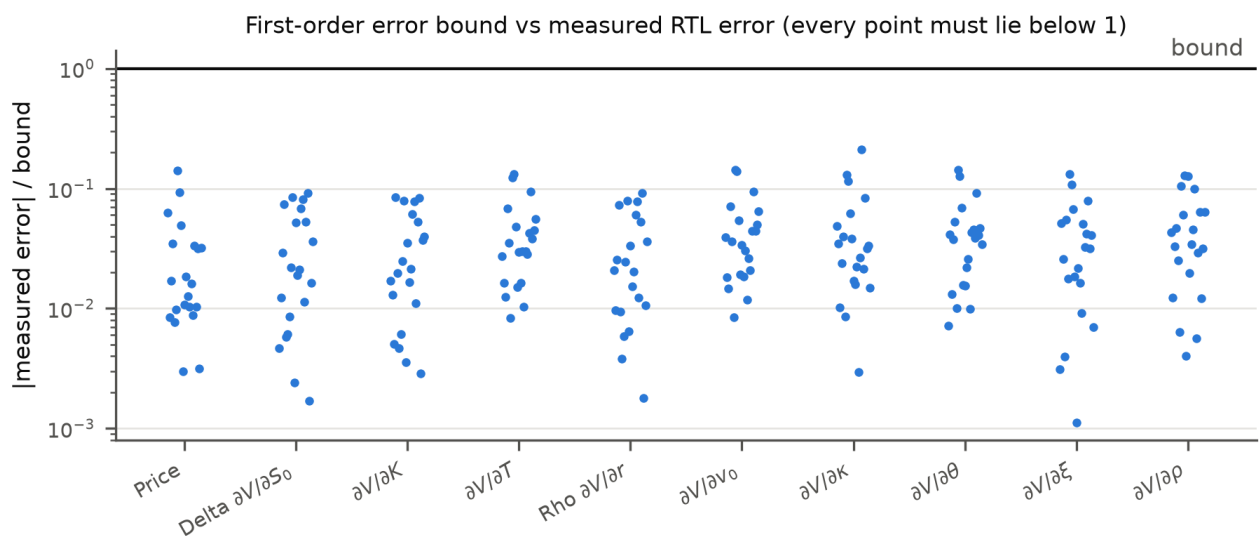


Figure 7. Generation 1: measured error as a fraction of the first-order bound, every output and case; all below 1.

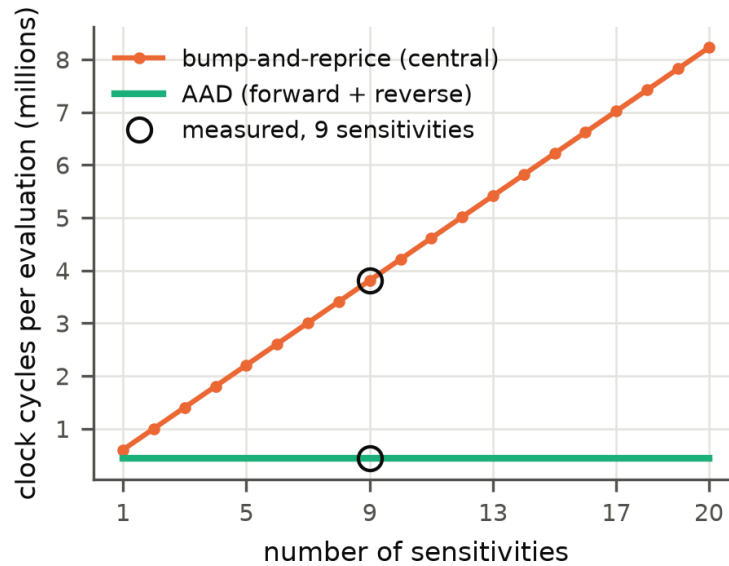


Figure 8. Generation 1: cycles against the number of Greeks, flat for AAD and linear for bump-and-reprice.

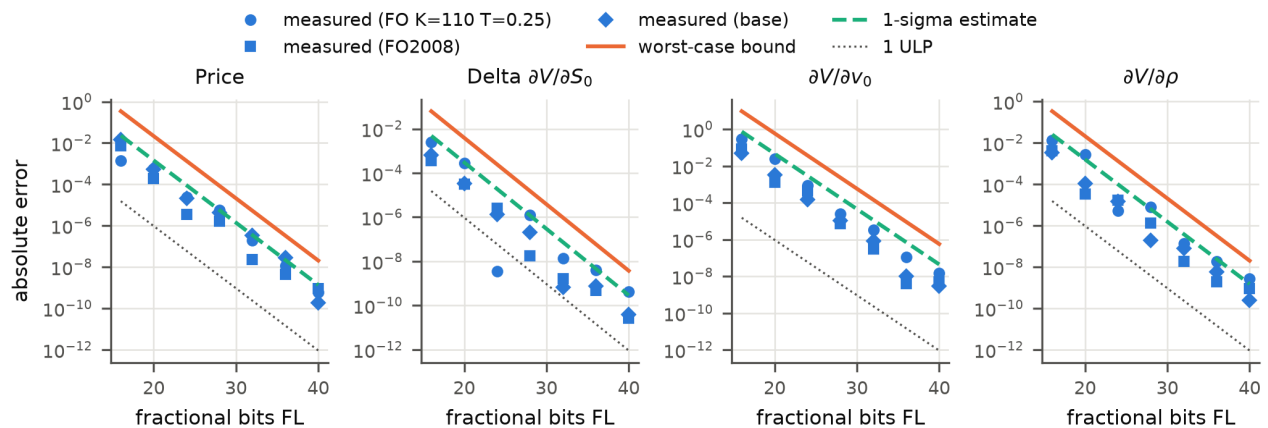


Figure 9. Generation 1: error against the number of fractional bits. The 1-sigma form of the bound predicts the measured error from 16 to 40 fractional bits.

7 Work done

The project ran from 15 August to 29 September 2026: 39 commits over 46 days.

Period	Milestone
15 Aug	Black-Scholes and Heston models, software AAD engine, MATLAB and C++ prototypes.
21 Aug – 9 Sep	Generation 1: hand-written FSM engine for Heston AAD in 64-bit fixed point; RTL validation report.
17 Sep	Fixed-point precision fixes, first-order error bound, bump-and-reprice baseline; the datapath generator (IR, emulator, modulo scheduler, RTL writer); fits the Zynq-7020.
22 – 24 Sep	Vivado implementation flow; viva study guide and a full course document.
26 – 27 Sep	AXI4-Lite register file, ZedBoard JTAG and processor designs; first complete route (68.6% LUT, ~79 MHz).
29 Sep	ZedBoard bitstream at 70 MHz; one vector then 50 cases bit-exact on silicon; calls through put-call parity.

Component	Size
Datapath generator (Python)	3,069 lines, 11 files
Generated Verilog	22,924 lines, 11 designs and wrappers
Hand-written Verilog (generation 1 and arithmetic library)	5,351 lines, 25 files
Validation (references, sweeps, figures, board report)	2,046 lines
Software models and AAD engine (Python)	2,359 lines
Vivado Tcl (implementation, board build and test)	683 lines
Documentation	3,806 lines
Automated verification	22 checks, all passing

Table 8. Size of the work, measured from the repository.

8 Limitations

- **79 MHz, not 100 MHz.** The engine misses 100 MHz by 2.6 ns; latencies are quoted at the measured Fmax.
- **The board bitstream is one change behind.** Silicon results used the design before put-call parity; the current RTL passes all simulation checks and awaits a rebuild.
- **Host steps run on the PC.** Setup and finish are Python, not C on the ARM; the processor-based system is scripted but not built.
- **Power is a vectorless estimate** at a 12.5% toggle rate; no activity-based figure or measurement yet.
- **Priority is stated "to our knowledge".** The literature check (section 2) cannot see unpublished industry work, and should be repeated in Google Scholar, IEEE Xplore and the ACM Digital Library before submission.
- **Bounded input domain:** $S_0 = 100$, K 60–150, T 0.1–3, r 0–0.1, v_0 and θ 0.005–0.25, κ 0.2–6, ξ 0.1–1, ρ –0.95–0.6. Outside it the hardware raises range_err.
- **The adjoint is hand-written.** The reverse sweep is written as operations in the IR (hardware/gen/heston.py) and verified bit-exact and against independent references, but no AD tool derives it. The paper should say "adjoint differentiation" in its claim, or the generator should derive the sweep.
- **Greeks hold the truncation range fixed** when differentiating, the standard COS convention; their distance from the model's true Greeks is part of the method error.

9 Next steps

	Step	Why	Status
1	Rebuild the ZedBoard bitstream; repeat the 50-case sweep	Bring the silicon result up to the current design	Ready: ~40 min build
2	Time a CPU software-AAD baseline	Biggest open risk: if a CPU core is close to 57.8 μ s, argue throughput and energy	Open, critical
3	Repeat the literature search before submission	Catch anything published since 29 Sep 2026	Done once
4	Close timing at 100 MHz	DSP output registers, pipelined carry chains	Open
5	Port host steps to C; build the processor design	No PC in the loop	Open
6	Activity-based power or board measurement	A defensible energy figure	Open
7	Merge into main	After step 1	Pending

Table 9. Next steps in order of value for the paper.

10 Conclusion

Reverse-mode AAD for the Heston model has been built as a fixed-point hardware datapath with no tape. Generated from one description, it computes the price and nine Greeks in 4,733 cycles, 1.04 pricings and 18.4 times fewer cycles than bump-and-reprice on the same hardware, within a guaranteed error bound for every output. It fits a Zynq-7020, meets timing on a ZedBoard at 70 MHz, and reproduced the emulator bit for bit on 50 cases in silicon. The immediate work is to rebuild the board with the put-call parity design and to give the latency a CPU comparison, which is now the main open risk.

Reproducing the results

`./verify_all.sh` (all 22 checks, about 4 minutes) · `python validation/run_gen_sweeps.py` (cycles, bump and accuracy data) · `python validation/make_figures.py` (figures and tables) · `zedboard/run_board.bat build | sweep (board) · python validation/board_report.py <log>` (board accuracy). Status dashboard: `dashboard.html`.